

Dokumentacja projektu: pustynia z obrazu w skali szarości

Ten dokument opisuje prosty projekt w Unity. Projekt generuje teren pustyni na podstawie tekstury w skali szarości, dodaje gorący wygląd sceny, efekt falowania powietrza, pył piaskowy, odbłask słońca i prosty kontroler FPS.

Kod jest celowo napisany prosto. Nie jest to duży profesjonalny system terenu. To wersja zrobiona tak, żeby dało się ją zrozumieć, pokazać w Inspectorze i wytłumaczyć na prezentacji.

Co robi projekt

W scenie jest obiekt `TerrainGenerator`. Ma komponent `DesertTerrainGenerator`, w którym można ustawić teksturę źródłową, rozmiar terenu, wysokość, rozdzielczość, poziom iso, materiał piasku i collider.

Po kliknięciu `Generate Terrain` skrypt tworzy dziecko o nazwie `Generated Desert Terrain`. Ten obiekt ma:

- ? `MeshFilter`
- ? `MeshRenderer`
- ? opcjonalny `MeshCollider`
- ? komponent `GeneratedDesertTerrain`
- ? komponent `DesertTerrainTile`

W scenie działa też `InfiniteDesertTerrainStreamer`. On kopiuje wygenerowany kafel wokół gracza i w kierunku patrzenia. Dzięki temu po dojściu do brzegu pojawiają się następne kafle, a dalekie kafle są usuwane albo wyłączane.

Jak tekstura szarości wpływa na teren

Tekstura w skali szarości jest traktowana jak mapa wysokości.

Biały kolor oznacza wyższy teren. Czarny kolor oznacza niższy teren. Szarości dają wartości pośrednie. Skrypt odczytuje jasność piksela i mnoży ją przez `heightMultiplier`.

Najważniejszy fragment jest w `DesertScalarField.cs`:

```
public float SampleDensity(Vector3 localPosition)
{
    float imageHeight = SampleHeight(localPosition.x, localPosition.z);
    return imageHeight - localPosition.y;
}
```

Ten kod porównuje wysokość z obrazka z wysokością punktu w świecie lokalnym. Jeżeli wynik jest dodatni, punkt jest pod powierzchnią terenu. Jeżeli wynik jest ujemny, punkt jest nad powierzchnią.

Drugi ważny fragment:

```
public float SampleHeight(float x, float z)
{
    float u = Mathf.InverseLerp(-terrainSize.x * 0.5f, terrainSize.x * 0.5f, x);
    float v = Mathf.InverseLerp(-terrainSize.y * 0.5f, terrainSize.y * 0.5f, z);
```

```

float brightness = SampleBrightness(u, v);

return brightness * heightMultiplier;
}

```

Tutaj pozycja **x** i **z** zostaje zamieniona na współrzędne UV od **0** do **1**. Potem skrypt pobiera jasność tekstury i robi z niej wysokość.

Co znaczy marching cubes w tym projekcie

Marching cubes to metoda tworzenia mesha z pola wartości. Najpierw mamy przestrzeń podzieloną na małe kostki. W rogach każdej kostki liczymy wartość pola. Jeżeli część rogów jest „pełna”, a część „pusta”, to znaczy, że przez kostkę przechodzi powierzchnia.

W tym projekcie jest uproszczona wersja tego pomysłu. Zamiast dużej tabeli wszystkich przypadków kostki, każda kostka jest dzielona na 6 małych tetraedrów. Tetraedr ma tylko 4 punkty, więc łatwiej policzyć, gdzie powierzchnia przecina jego krawędzie.

Fragment z **DesertMarchingCubes.cs**:

```

for (int tetrahedron = 0; tetrahedron < Tetrahedra.GetLength(0); tetrahedron++)
    PolygonizeTetrahedron(cornerPositions, cornerDensities, tetrahedron, isoLevel, builder);

```

Ten fragment pokazuje, że każda kostka jest rozbijana na prostsze bryły. Dla każdej z nich wykonywane jest osobne tworzenie trójkątów.

Najważniejsza decyzja, czy punkt jest wewnątrz terenu:

```

inside[i] = densities[i] >= isoLevel;

```

Jeżeli wartość pola jest większa albo równa **isoLevel**, punkt jest traktowany jako część terenu. **isoLevel** pozwala przesuwac granicę powierzchni.

Jak powstają trójkąty mesha

W **PolygonizeTetrahedron** skrypt sprawdza krawędzie tetraedru. Jeżeli jedna końcówka krawędzi jest pełna, a druga pusta, to powierzchnia przecina tę krawędź.

```

if (inside[a] == inside[b])
    continue;

crossings[crossingCount] = Interpolate(
    positions[a],
    positions[b],
    densities[a],
    densities[b],
    isoLevel);
crossingCount++;

```

Interpolate znajduje dokładne miejsce na krawędzi, gdzie wartość pola przechodzi przez **isoLevel**. Potem z tych punktów przecięcia powstaje jeden albo dwa trójkąty.

```

AddOrientedTriangle(crossings[0], crossings[1], crossings[2], normalHint, builder);

if (crossingCount == 4)
    AddOrientedTriangle(crossings[0], crossings[2], crossings[3], normalHint, builder);

```

Jeżeli przecięcia są 3, powstaje jeden trójkąt. Jeżeli przecięcia są 4, powstają dwa trójkąty.

Główny przepływ generowania

Najważniejsza metoda to `GenerateTerrain()` w `DesertTerrainGenerator.cs`.

```
DesertScalarField field = new DesertScalarField(
    grayscaleSourceImage,
    terrainSize,
    heightMultiplier,
    makeEdgesTileable);

Mesh mesh = DesertMarchingCubes.BuildMesh(
    field,
    terrainSize,
    heightMultiplier,
    resolution,
    isoLevel,
    sandTextureTiling,
    makeEdgesTileable);
```

Najpierw tworzony jest obiekt pola wartości, czyli `DesertScalarField`. Potem `DesertMarchingCubes.BuildMesh()` przechodzi po voxelach i buduje mesh.

Po stworzeniu mesha generator tworzy nowy obiekt w scenie:

```
GameObject terrain = new GameObject(generatedObjectName);
terrain.transform.SetParent(transform, false);
terrain.AddComponent<GeneratedDesertTerrain>();

MeshFilter meshFilter = terrain.AddComponent<MeshFilter>();
MeshRenderer meshRenderer = terrain.AddComponent<MeshRenderer>();
meshFilter.sharedMesh = mesh;
```

Dzięki temu wynik generowania jest normalnym obiektem Unity. Można go zobaczyć w hierarchii, ma renderera i można do niego dodać collider.

Collider jest dodawany tylko wtedy, gdy w Inspectorze jest włączone `Generate Collider`:

```
if (generateCollider)
{
    MeshCollider meshCollider = terrain.AddComponent<MeshCollider>();
    meshCollider.sharedMesh = mesh;
}
```

Jak działa przycisk w Inspectorze

W `DesertTerrainGenerator` jest atrybut:

```
[ContextMenu("Generate Terrain")]
public void GenerateTerrain()
```

Dzięki temu metodę można uruchomić z menu komponentu. Dodatkowo skrypt `DesertTerrainGeneratorEditor.cs` dodaje zwykły przycisk w Inspectorze. Dla użytkownika wygląda to prosto: zaznacza `TerrainGenerator`, ustawia parametry i klika `Generate Terrain`.

Ustawienia w Inspectorze

`Grayscale Source Image` to obraz, który steruje wysokością terenu.

`Terrain Size` ustawia rozmiar kafła terenu w osi X i Z.

`Height Multiplier` ustawia maksymalną wysokość.

Resolution ustawia gęstość siatki voxelowej. Większa wartość daje więcej szczegółów, ale tworzy więcej trójkątów.

Iso Level przesuwa granicę powierzchni. Najczęściej może zostać przy 0.

Make Edges Tileable wygładza próbki przy krawędziach, żeby kafle terenu lepiej do siebie pasowały.

Desert Material to materiał pustyni.

Sand Texture to tekstura piasku.

Generate Collider dodaje **MeshCollider**, żeby gracz mógł chodzić po terenie.

Clear Old Generated Terrain usuwa stary wygenerowany teren przed stworzeniem nowego.

Jak działa zapętlanie terenu

Sam generator tworzy jeden kafeł. Za nieskończone wrażenie terenu odpowiada **InfiniteDesertTerrainStreamer.cs**.

Streamer sprawdza, na którym kafle stoi gracz:

```
Vector2Int center = GetTileCoordinate(target.position);
Vector3 lookDirection = GetMainDirection();
HashSet<Vector2Int> desiredTiles = BuildDesiredTiles(center, lookDirection);
```

center to kafeł pod graczem. **lookDirection** to kierunek patrzenia kamery. **desiredTiles** to lista kafli, które powinny istnieć.

Streamer zawsze zostawia bezpieczny kwadrat wokół gracza:

```
int localSafetyRadius = Mathf.Max(safetyRadius, Mathf.Min(sideTiles, 3));
AddSquare(desiredTiles, center, localSafetyRadius);
AddDirectionalTiles(desiredTiles, center, lookDirection);
```

To chroni przed sytuacją, gdzie gracz nagle zobaczy dziurę pod sobą. Potem dodawane są kafle w kierunku patrzenia.

Kierunkowe dodawanie kafli działa tak:

```
float forwardDistance = Vector2.Dot(offset, forward);
float sideDistance = Mathf.Abs(Vector2.Dot(offset, right));

if (forwardDistance < -behindTiles || forwardDistance > forwardTiles)
    continue;

if (sideDistance <= allowedSideDistance)
    desiredTiles.Add(center + new Vector2Int(x, y));
```

Skrypt bierze kafle przed graczem i po bokach kierunku patrzenia. Kafle za graczem są ograniczone przez **behindTiles**, więc dalekie, niepotrzebne kafle mogą zniknąć.

Kafle, które nie są już potrzebne, są usuwane:

```
CreateMissingTiles(desiredTiles);
RemoveUnwantedTiles(desiredTiles);
```

W praktyce daje to efekt terenu, który pojawia się tam, gdzie gracz idzie albo patrzy.

Jak zmniejszono widoczne krawędzie kafli

Widoczne krawędzie mogą powstać z kilku powodów. Najważniejsze w tym projekcie to różne próbki wysokości na brzegach, osobne normalne mesha oraz tekstura piasku zaczynająca się od nowa na każdym kafle.

Pierwsza poprawka jest w `DesertScalarField.cs`. Przy włączonym `Make Edges Tileable` jasność jest mieszana z odbitymi próbkami:

```
float bottom = Mathf.Lerp(SampleRawBrightness(u, v), SampleRawBrightness(1f - u, v), xBlend);
float top = Mathf.Lerp(SampleRawBrightness(u, 1f - v), SampleRawBrightness(1f - u, 1f - v),
    xBlend);

return Mathf.Lerp(bottom, top, yBlend);
```

To nie robi idealnej matematycznej pętli każdej tekstury, ale mocno zmniejsza ostre różnice na krawędziach.

Druga poprawka jest w normalnych mesha:

```
if (smoothTileEdges)
    SmoothTileEdgeNormals(mesh, terrainSize);
```

`SmoothTileEdgeNormals` uśrednia normalne na przeciwległych krawędziach kafła i lekko miesza je w stronę `Vector3.up`. Dzięki temu światło nie tworzy tak ostrej pionowej linii między kafłami.

Trzecia poprawka jest w shaderze piasku. `WorldSpaceSand.shader` nie bierze UV z mesha, tylko pozycję świata:

```
float2 worldUv = input.positionWS.xz * _WorldTextureScale;
half4 sand = SAMPLE_TEXTURE2D(_BaseMap, sampler_BaseMap, worldUv) * _BaseColor;
```

Ponieważ tekstura zależy od pozycji w świecie, a nie od lokalnego UV kafła, wzór piasku przechodzi przez granice kafli płynnie.

Materiał pustyni

Materiał `DesertSand.mat` używa shadera `Pustynia/WorldSpaceSand`. Ma ciepły kolor piasku i teksturę `ProceduralSand.png`.

W shaderze jest proste oświetlenie:

```
half3 softenedNormal = normalize(lerp(half3(0, 1, 0), normalize(input.normalWS),
    _NormalStrength));
Light mainLight = GetMainLight();
half directLight = saturate(dot(softenedNormal, mainLight.direction));
half lightAmount = _AmbientStrength + directLight * (1.0h - _AmbientStrength);
```

Normalna jest trochę wygładzana, żeby pustynia nie wyglądała jak poszarpany plastik. `AmbientStrength` dodaje światło bazowe, więc cienie nie są zbyt czarne.

Wygląd gorącej pustyni

Scena ma mocne światło kierunkowe `Desert Sun`, ciepły skybox, mgłę piaskową i `Hot Desert Global Volume`.

Volume dodaje:

? ciepły balans bieli

- ? lekkie podbicie koloru
- ? delikatny bloom
- ? winietę
- ? małą aberrację chromatyczną

Chodzi o to, żeby scena wyglądała na gorącą i suchą, ale nadal była czytelna.

Efekt gorącego powietrza

Za falowanie powietrza odpowiada `DesertHeatHazeOverlay.cs` i shader `HeatHazeOverlay.shader`.

Skrypt tworzy przed kamerą płaski quad z materiałem. Shader próbuje odczytać kolor sceny i przesuwa UV falami:

```
float waveX = sin((screenUv.y * _Scale + time) * 6.2831853);
float waveY = sin((screenUv.x + screenUv.y) * _Scale * 0.45 - time * 0.8) * 6.2831853);

float2 offset = float2(waveX + smallRipple * 0.35, waveY) * _Strength;
half3 sceneColor = SampleSceneColor(saturate(screenUv + offset));
```

To daje prosty efekt drgania obrazu. Nie jest to zaawansowana symulacja temperatury, tylko ekranowy trik wizualny.

Lens flare

`DesertLensFlare.cs` tworzy kilka prostych kół odbłasku. Kół nie jest dużo, żeby efekt nie zasłaniał sceny.

Obiekt flary ustawia się w kierunku słońca:

```
Vector3 sunDirection = -sunTransform.forward;
transform.position = cameraToUse.transform.position + sunDirection.normalized * sunDistance;
```

Potem skrypt liczy pozycję słońca w widoku kamery i układa koła na linii od słońca do środka ekranu:

```
Vector2 sunViewport = new Vector2(viewportPosition.x, viewportPosition.y);
Vector2 sunToCenter = new Vector2(0.5f, 0.5f) - sunViewport;
Vector2 viewportOffset = sunToCenter * circles[i].linePosition;
```

To jest prosty, czytelny efekt do projektu. Nie udaje prawdziwego systemu optycznego.

Pył piaskowy

`Sand Dust Atmosphere` używa `ParticleSystem`. Cząstki są małe i mają materiał pyłu. Mają dawać wrażenie drobnego piasku w powietrzu, a nie wielkich kwadratów.

Pył działa razem z mgłą Unity. Mgła ma ciepły kolor piasku, więc dalszy teren staje się mniej kontrastowy i bardziej pustynny.

Prosty kontroler FPS

`SimpleDesertFpsController.cs` używa `CharacterController`. Ma tylko podstawowe funkcje:

- ? chodzenie
- ? skakanie

? obrót myszką

? wykrywanie ziemi

Ruch jest liczony z klawiszy WASD albo strzałek:

```
Vector3 inputDirection = transform.right * moveInput.x + transform.forward * moveInput.y;
Vector3 targetVelocity = inputDirection * walkSpeed;
horizontalVelocity = Vector3.Lerp(horizontalVelocity, targetVelocity, acceleration *
    Time.deltaTime);
```

Myszka obraca gracza w poziomie i kamerę w pionie:

```
transform.Rotate(Vector3.up * (lookInput.x * mouseSensitivity), Space.World);
pitch = Mathf.Clamp(pitch - lookInput.y * mouseSensitivity, minPitch, maxPitch);
playerCamera.transform.localRotation = Quaternion.Euler(pitch, 0f, 0f);
```

Skok liczy prędkość pionową z wysokości skoku i grawitacji:

```
verticalVelocity.y = Mathf.Sqrt(jumpHeight * -2f * gravity);
```

Wykrywanie ziemi

Kontroler nie używa tylko `CharacterController.isGrounded`, bo przy nierównym meshu może to być niestabilne. Zamiast tego robi `SphereCast` pod graczem:

```
int hitCount = Physics.SphereCastNonAlloc(
    origin,
    radius,
    Vector3.down,
    groundHits,
    distance,
    groundMask,
    QueryTriggerInteraction.Ignore);
```

Potem skrypt filtruje wyniki. Ignoruje własne collidery, ruchome rigidbody i zbyt strome powierzchnie:

```
if (hit.collider == null || hit.collider.transform.IsChildOf(transform))
    continue;

if (Vector3.Angle(hit.normal, Vector3.up) > controller.slopeLimit)
    continue;
```

Dzięki temu gracz stabilniej chodzi po wygenerowanym terenie.

Jest też `coyoteTime`, czyli krótki czas po zejściu z krawędzi, kiedy nadal można skoczyć:

```
coyoteTimer = isGrounded ? coyoteTime : coyoteTimer - Time.deltaTime;
```

To sprawia, że chodzenie i skakanie są mniej nerwowe.

Skrypty w projekcie

`DesertTerrainGenerator.cs` to główny komponent generatora. Trzyma ustawienia z Inspector'a i tworzy `GameObject` z meshem.

`DesertScalarField.cs` czyta teksturę w skali szarości i zwraca wartość pola dla punktu 3D.

`DesertMarchingCubes.cs` buduje mesh z voxelowego pola. Używa uproszczenia z tetraedrami.

`GeneratedDesertTerrain.cs` jest prostym znacznikiem, który mówi, że obiekt został wygenerowany.

`DesertTerrainTile.cs` przechowuje współrzędne kafła terenu.

`InfiniteDesertTerrainStreamer.cs` kopiuje kafle terenu wokół gracza i w kierunku patrzenia.

`DesertTerrainGeneratorEditor.cs` dodaje przyciski w Inspectorze.

`SimpleDesertFpsController.cs` pozwala chodzić po scenie.

`DesertHeatHazeOverlay.cs` dodaje ekranowy efekt falowania gorącego powietrza.

`DesertSunGlare.cs` ustawia prosty odbłask słońca.

`DesertLensFlare.cs` tworzy koła lens flare.

`DesertSceneSetup.cs` tworzy demo sceny, materiały, tekstury, volume, światło, gracza i efekty.

`WorldSpaceSand.shader` odpowiada za materiał piasku bez widocznego restartu UV na każdym kafłu.

`HeatHazeOverlay.shader` odpowiada za falowanie gorącego powietrza.

`SunGlare.shader` odpowiada za miękki odbłask słońca i koła lens flare.

Instrukcja użycia

1. Otwórz scenę `Assets/Scenes/DesertDemo.unity`.
2. Zaznacz obiekt `TerrainGenerator`.
3. W polu `Grayscale Source Image` ustaw teksturę w skali szarości.
4. Ustaw `Terrain Size`, `Height Multiplier`, `Resolution` i `Iso Level`.
5. Ustaw materiał `DesertSand.mat`.
6. Zostaw włączone `Generate Collider`, jeśli gracz ma chodzić po terenie.
7. Kliknij `Generate Terrain`.
8. Uruchom Play Mode.
9. Kliknij w Game View, żeby zablokować kursor.
10. Chodź klawiszami WASD, skacz spacją i rozglądaj się myszką.

Najważniejszy kod do omówienia na prezentacji

Najpierw można pokazać `DesertTerrainGenerator.GenerateTerrain()`, bo tam widać cały przepływ: ustawienia, pole wartości, budowanie mesha i tworzenie obiektu.

Potem warto pokazać `DesertScalarField.SampleDensity()`, bo to tłumaczy, jak obraz staje się wysokością.

Następnie można pokazać `DesertMarchingCubes.PolygonizeTetrahedron()`, bo tam widać ideę marching cubes: punkty pełne, punkty puste, przecięcia krawędzi i trójkąty.

Na końcu warto pokazać `InfiniteDesertTerrainStreamer.BuildDesiredTiles()`, bo to tłumaczy, czemu teren wydaje się nieskończony.

Jeżeli prowadzący zapyta o efekt krawędzi kafli, najlepiej powiedzieć, że projekt używa trzech prostych poprawek: mieszania próbek przy brzegach tekstury, wygładzania normalnych na krawędziach mesha i shadera piasku opartego o pozycję świata.

Krótkie podsumowanie

Projekt generuje pustynny teren z jasności obrazu. Uproszczony marching cubes tworzy mesh z pola wartości. Streamer kopiuje kafle, żeby teren nie kończył się pod graczem. Materiał, volume, mgła, pył, heat haze i lens flare tworzą gorący pustynny klimat. Prosty kontroler FPS pozwala przejść się po wygenerowanej scenie i sprawdzić teren w praktyce.